

— A PORTFOLIO PROJECT · 2026

BrickLens. Trustworthy fault detection over a building knowledge graph.

A Brick-modelled building, validated in RDF and served from a property graph, with deterministic detectors that localise every alert to the equipment responsible — and prove it.

Vinay Gautam · Trondheim

rdflib · Brick 1.3 · Neo4j · DuckDB · FastAPI · GraphQL

A BMS raises alarms. Almost none of them say **where** the fault is, or **why**.

Alarms without context

Conventional fault detection fires on a threshold and hands the operator a tag name. Which zone, which unit, which of the three things upstream? That work is still manual.

Data in silos

The BMS, the sensor network and the building model each know one part of the picture. Nothing joins them, so every analysis starts from CSV exports.

IEQ is now a compliance matter

The revised EPBD makes indoor environmental quality — CO₂, temperature, humidity, ventilation — a monitored requirement, not a design aspiration.

Intent: build the smallest system that detects a fault, says where it is, shows why, and can prove how often it is right.

Context is not a pile of CSVs. It is a model of the building.

Brick gives every zone, unit, damper and sensor a stable identity and a typed relationship: what feeds what, what is part of what, what measures what. Three questions become graph queries:

- Which zone is this CO₂ sensor in?
- Which equipment serves that zone, and what is upstream of it?
- Which other points share that equipment — and are they healthy?

Those three answers are exactly what fault localisation needs. Without the graph they are tribal knowledge; with it they are one traversal.

```
demo building · 54 entities · 146 relations
```

```
Building → Floor → 4 HVAC zones
```

```
AHU-1 —FEEDS→ VAV-1...4 —FEEDS→ zones
```

```
VAV —HAS_PART→ damper
```

```
zone / equipment —HAS_POINT→ 34 points
```

```
CO2 · temperature · setpoint · humidity · damper  
cmd/pos · reheat · valve cmds · fan status ·  
power
```

The rules live in a validation layer, **not in a model's judgement.**

Faithful explanations

Every reason attached to a suspect is the term that produced its score. The explanation is the computation, not a story written afterwards.

Falsifiable

A rule that is wrong is wrong the same way every time. Run it on labelled data, count, fix once, add a test.

Runs on every point, every interval

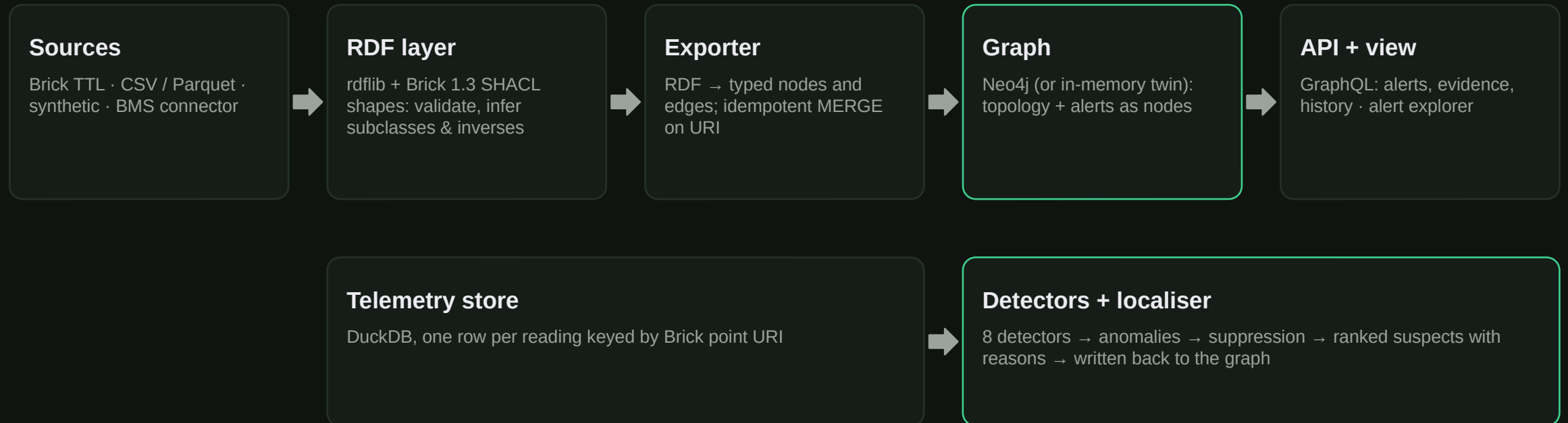
34 points at five-minute resolution is trivial; 30 000 across a campus is still trivial. Nothing needs to be pre-selected for a context window.

Stable and self-contained

Behaviour is pinned to a code version and a config file with rationales. No model deprecation changes what the building reports; no free text enters the reasoning path.

A language model can sit on top of a validated alert — to narrate, map tags, or answer questions. It should never be the thing that decides a fault exists.

Validate in RDF. Serve from a property graph. Bind the time series on the URI.



The model is never edited in the property graph: a change means re-validate and re-export. Detectors never touch raw files; the API never touches RDF.

Each representation does the job it is built for.

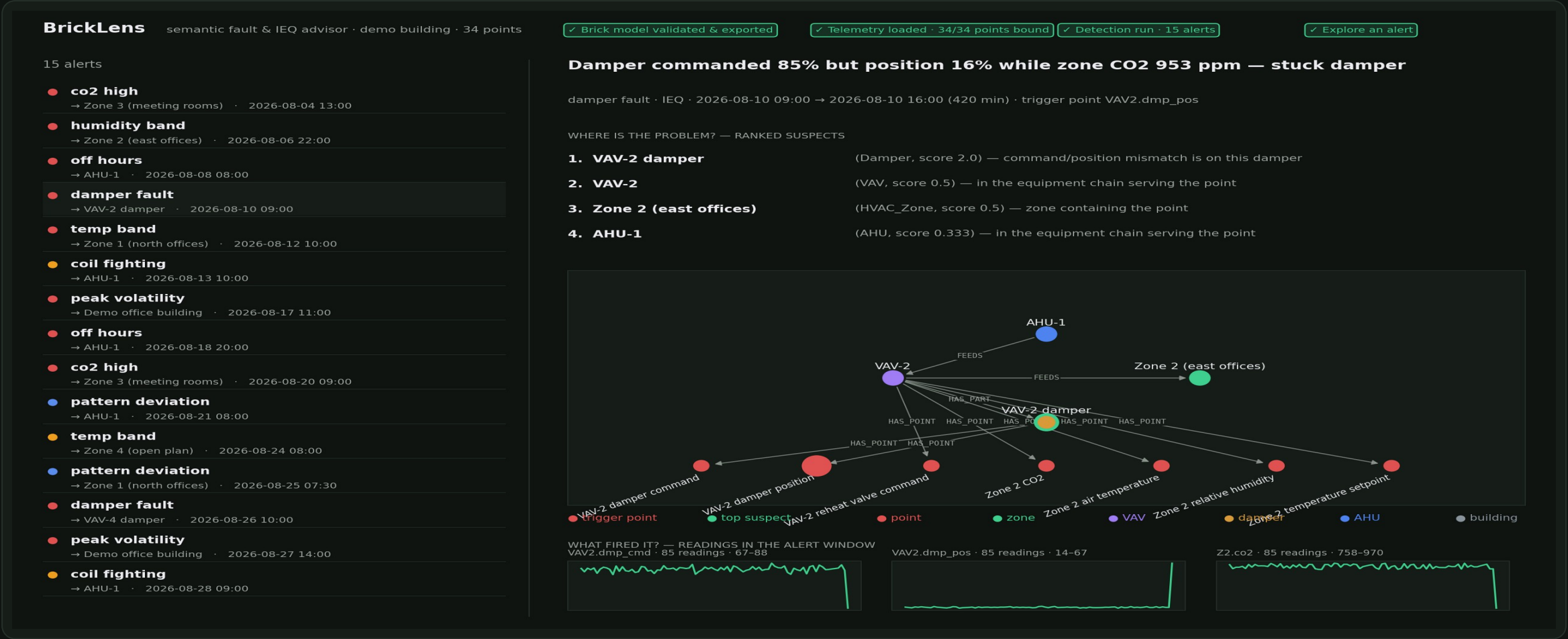
Concern	RDF (rdflib, Brick shapes)	Property graph (Neo4j / networkx)
Is the model valid Brick?	SHACL validation against the vendored ontology; a malformed model is rejected before export	—
Is a zone temperature sensor also a Temperature_Sensor and a Point?	Subclass closure and inverse relations materialised once	Carried as a classes list on every node
What serves this point, up to four hops?	Property paths plus code	One variable-length Cypher path, nearest first
Everything that has implicated AHU-1 this month?	—	Alerts are nodes: MATCH (a:Alert)-[:IMPLICATES]->(e {uri})
Show me the evidence	JSON-LD view keyed by Brick URI	Node/edge subgraph, drawn in the explorer or Neo4j Browser

Node label = most specific Brick class · relationship types = Brick predicates (FEEDS, HAS_PART, HAS_POINT...) · the mapping is forty lines and part of the documented contract.

Eight detectors. Each declares the Brick point classes it needs, and skips itself where they are absent.

Alert	Family	Rule	Localised to
co2_high	IEQ	> 1000 ppm for 30 min	zone → VAV → AHU
temp_band	IEQ	outside setpoint ± 2 °C while occupied	zone → VAV
humidity_band	IEQ	outside 30–60 % RH for 60 min	zone → AHU
damper_fault	IEQ / FDD	commanded > 60 %, position lags > 10 %, CO ₂ > 900	damper → VAV → AHU
off_hours	energy	> 2 kW outside the schedule for 60 min	AHU
coil_fighting	energy	heating and cooling both > 10 % for 15 min	AHU or VAV
peak_volatility	energy	> 3σ above the same-slot baseline	building
pattern_deviation	statistical	weekly-profile residual, $ z > 4$ for 60 min	point's owner

One screen answers three questions: where, why, and what fired it.



From a point to a ranked list of suspects, in three graph queries.

1 Chain

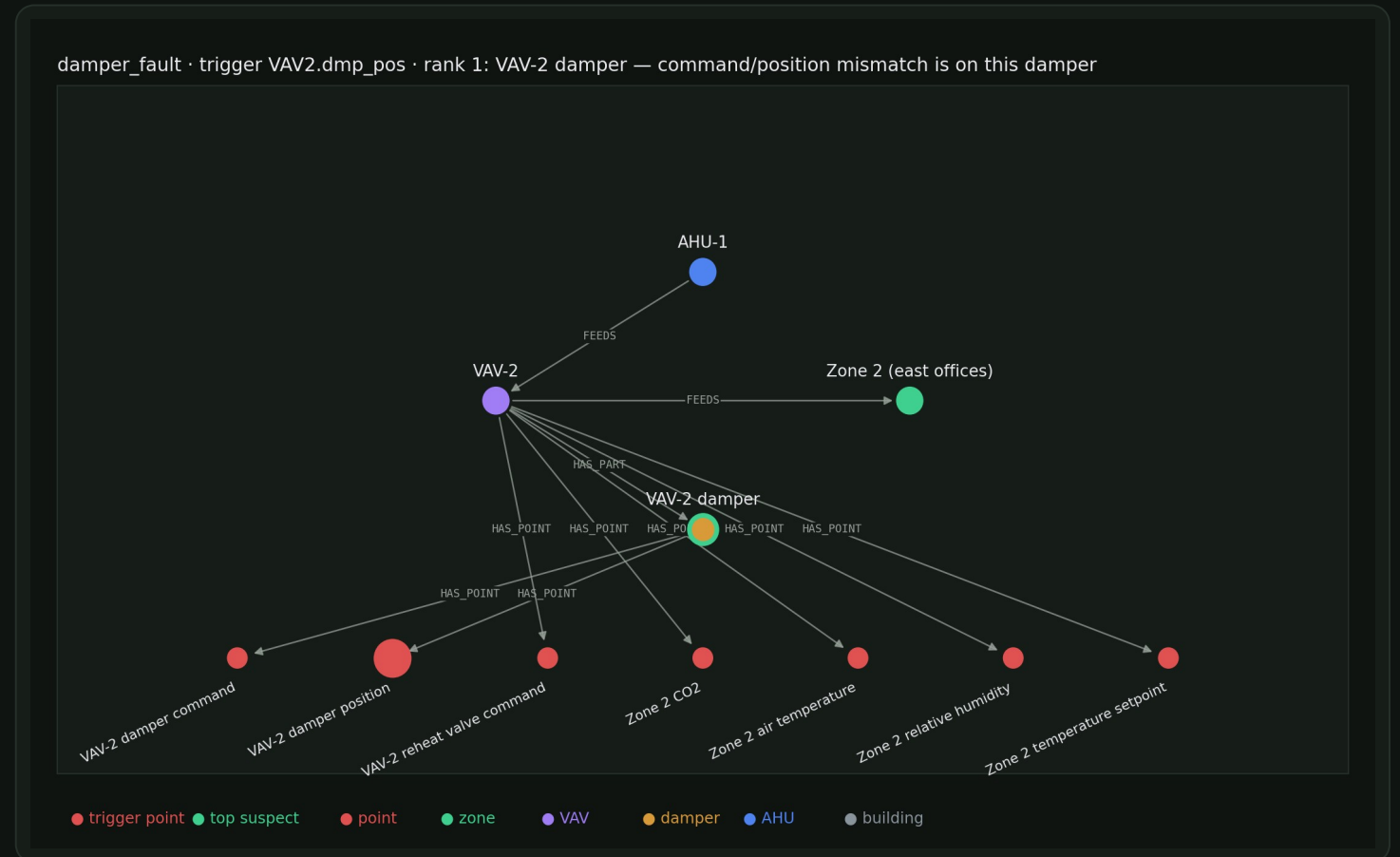
Walk `IS_POINT_OF · IS_PART_OF · IS_FED_BY` up to four hops: damper → VAV → AHU → building, nearest first.

2 Siblings

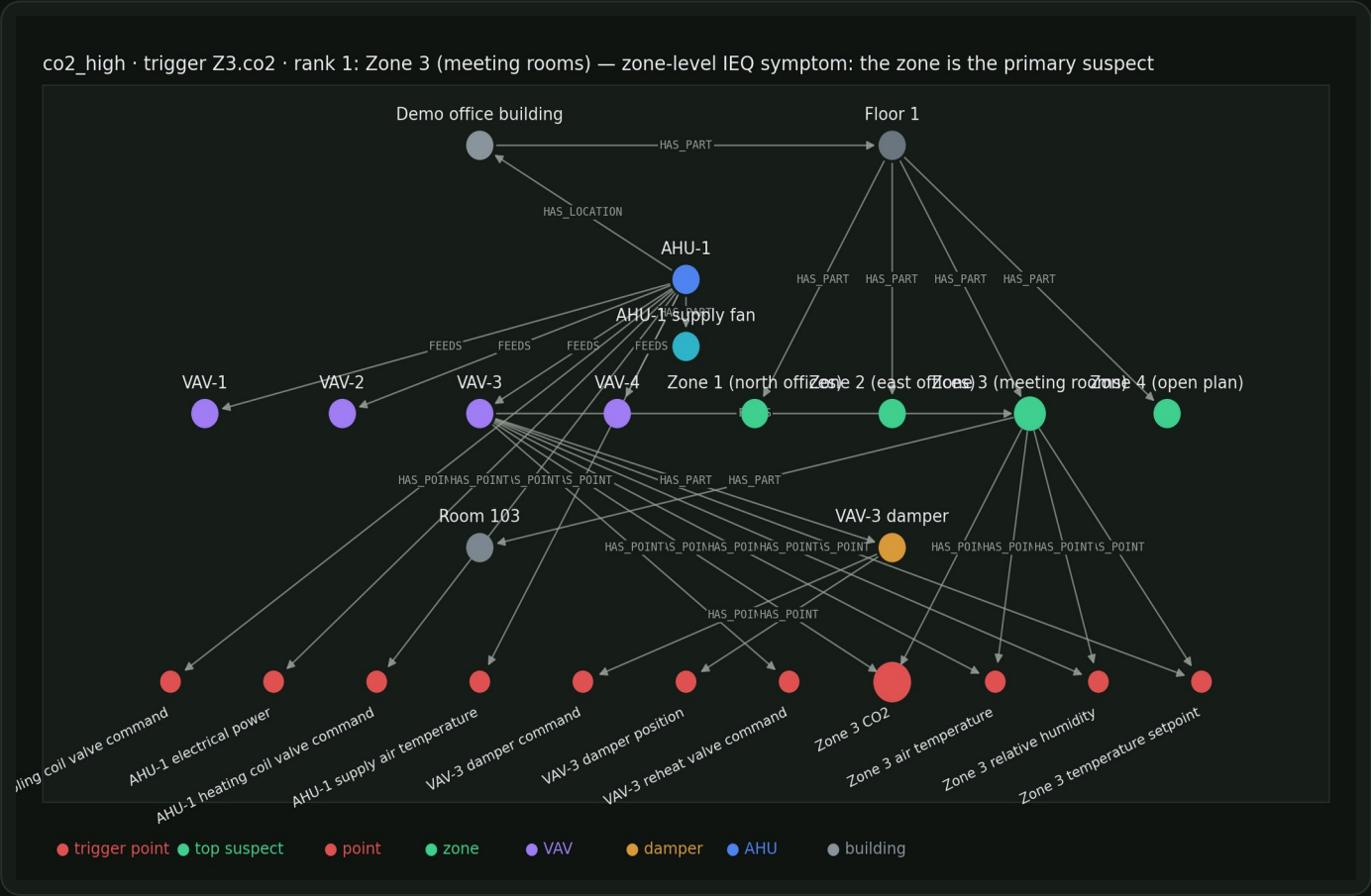
Other points on the same owner that are anomalous in the same window. Two or more: the unit, not the sensor.

3 Score, with a reason

Distance prior $1/(1+\text{hops})$, plus bonuses that encode how an engineer reads a symptom. Every bonus returns its own one-line reason.



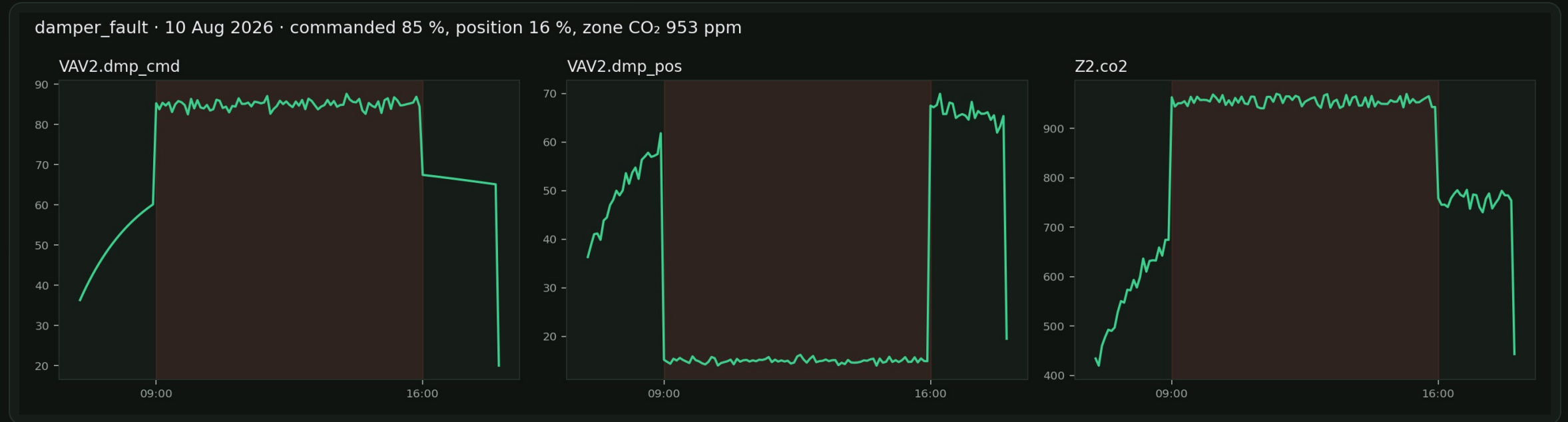
The graph knows the topology. The scoring knows how a symptom is read.



Symptom	Rank 1
CO ₂ , temperature, humidity in a zone	the zone — then its terminal unit
command / position mismatch	the damper
main-meter demand spike	the building
≥ 2 anomalous sibling points	the shared unit
same symptom in ≥ 2 zones fed by one AHU	the AHU

Pure shortest-path ranking would put the VAV ahead of the zone for an occupancy-driven CO₂ rise. That is wrong for the reader, so the convention says so — and says why.

Thresholds with dwell times, baselines with floors — and a rationale for every number.



Dwell time. CO₂ must exceed 1000 ppm for 30 minutes; a meeting that ends is not a fault.

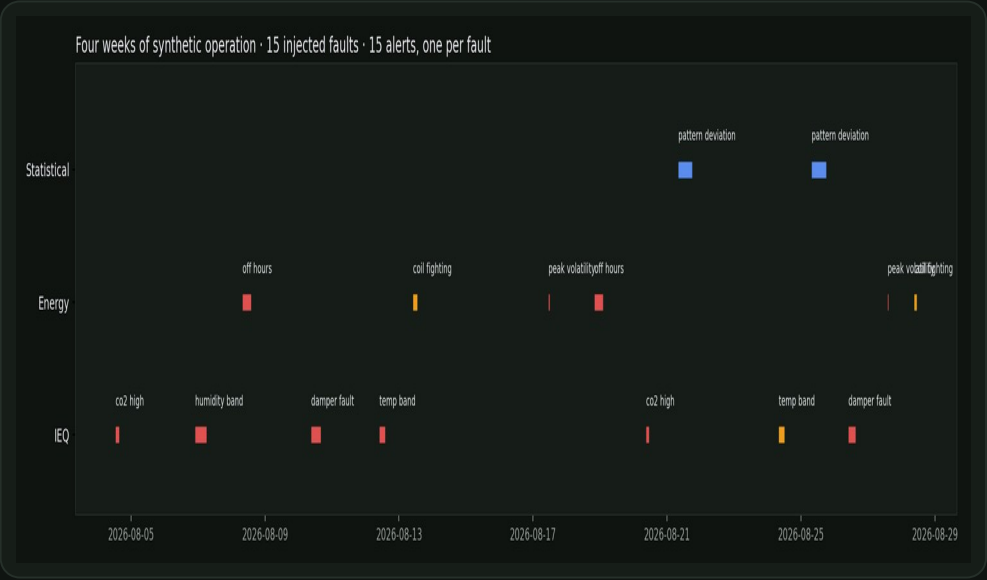
Sigma floor. Demand baselines never scale below 4 kW: an 8 kW AHU restart at night is not a spike.

Suppression. A statistical alert already explained by a rule on the same zone and window is dropped.

Detection quality is measured, gated in CI, and stable across noise seeds.

Metric	Definition	Result
Precision	alerts intersecting a same-type labelled fault	1.00 (15 / 15)
Recall	faults covered ≥ 50 % by same-type alerts	1.00 (15 / 15)
Localisation accuracy	rank-1 suspect equals the fault's target	1.00
False alerts per zone-week	$FP \div (zones \times weeks)$	0.0
Noise-seed stability	targets hold across six seeds	pass

Ground truth: four weeks of synthetic telemetry with 15 scripted faults across all eight alert types. Perfect scores are the expected baseline here, not a claim of generalisation — what they establish is one alert per fault, correct localisation for every type, and a build that fails when a threshold change breaks any of it.



The first statistical detector scored perfectly — until the noise changed.

What happened

The Isolation Forest variant reached 15 / 15 on the synthetic run. An unrelated edit shifted the random noise sequence — same faults, different noise — and it then flagged spurious humidity hours and covered one hour of a ten-hour sensor drift.

What it means

A detector whose verdict depends on the noise realisation cannot gate a build. Plausible results are not the same as reproducible ones.

What replaced it

A weekly-profile residual: median and MAD per 15-minute slot, learned from the first week, $|z| > 4$ for 60 minutes. Deterministic, explainable in a sentence, 15 / 15 across six seeds. Isolation Forest stays as an option; a test guards the stability.

Recorded as a decision with its evidence, alongside every threshold change. Knowing when not to use machine learning is part of the method.

Portability is engineered, **not asserted**.

Detectors declare what they need

Required Brick point classes, resolved through the inferred hierarchy. A building without zone CO₂ runs the six detectors it can and reports the two it cannot.

One ingestion seam

TelemetrySource: two methods. CSV, Parquet and synthetic ship; a public AHU dataset and a BMS REST stub are documented. Unit normalisation happens there, never in a detector.

The graph is drop-in

Brick classes as labels, Brick predicates as relationship types, the URI as the key. Custom properties such as time-series ids and value ranges attach without altering the ontology.

Coverage is visible

Ingestion status reports points without data and series without a point. Nothing is silently assumed.

A validated, ranked, evidence-backed alert is the right thing to build everything else on.

Real signals

Run the public AHU field dataset; then naturally occurring faults with the people who fixed them.

Provenance and known gaps

Source, method and verification on every node; unevaluated checks surfaced as items, with declared defaults.

A language model at the edges

Map BMS tags to Brick; narrate and recommend on top of the alert; answer questions over the graph. Never decide.